tidySTL: Exposing Implicit Semantics in STL Evaluation

Sota Sato[1] and Masaki Waga[2]

[1] National Institute of Advanced Industrial Science and Technology (AIST)
[2] Kyoto University

Abstract. Signal Temporal Logic (STL) is a specification language with quantitative robust semantics, used across runtime monitoring, falsification, and learning-enabled control. Yet capable tools for evaluating the robust semantics are typically organized around particular applications: a runtime monitor here, a falsification backend there. We argue that this fragmentation is largely not accidental. The robust semantics has historically been realized through signal-processing pipelines, in which low-level choices (signal representation, sampling, interpolation) materially affect the robustness values. Together these choices act as an *implicit semantics*, resulting in a patchwork of tools that are not directly comparable and not easily repurposed outside their original application contexts. We present tidySTL, a Python library built around an explicit computation DAG, which decouples logical syntax from these implicit choices and exposes extension points where they vary. We validate tidySTL with a systematic test suite and cross-check it against existing STL tools, revealing discrepancies that flip verdicts and can mislead downstream falsification. We also demonstrate its extensibility by expressing representative semantics via localized changes to the evaluator layer.

1 Introduction

Signal Temporal Logic (STL) is a standard formalism for specifying temporal properties of real-valued signals [16,10]. Its quantitative *robustness* semantics measures how strongly a behavior satisfies or violates a specification, and underpins tasks in runtime monitoring [2], falsification [1], and learning-enabled design [20].

That semantics, though, is written for an idealized signal: continuous, infinitely long, and defined at every instant. The signals an evaluator actually receives are finite and sampled, and closing the gap demands decisions the formula is silent about—how a signal behaves between two samples, for instance, or what a temporal window sees past the last one. We call these decisions *implicit semantics*: they are absent from the STL formula yet they shift the robustness value computed for it.

These decisions are easy to overlook, yet their consequences are not. Two tools that resolve them differently can disagree on the same formula and the same trace, with the difference tracing back to a silently adopted choice rather

than to anything the specification says—and a downstream search or monitor runs to completion with no indication that its output rests on a convention (§4.1).

Existing tools reach evaluation through an application: a monitor, a falsifier, or a learning library computes robustness as a means to its own end, and the implicit choices are settled wherever that pipeline happens to settle them. We take the opposite stance and treat evaluation itself as the object of interest. tidySTL is built evaluation-first: the logical core of STL stays fixed behind a stable interface, and the implicit choices live below it as explicit, interchangeable backends rather than hard-coded assumptions.

Building a tool around evaluation rather than an application invites the question of what it is good for. This paper makes the following contributions.

- We identify STL robustness evaluation as a common but often implicit computational layer shared across monitoring, falsification, and related applications, and we organize its implicit choices into explicit decision points (§2).
- We present tidySTL,[3] a lightweight Python library that makes this layer explicit: formula structure, signal data, and evaluation are separated so that semantic choices attach as a switchable backend below a stable public API (§3).
- We validate tidySTL with a systematic regression suite and a cross-tool comparison against Breach, RTAMT, and STLCG++. The comparison reveals that evaluator-level choices can cause observable discrepancies under the same nominal input (§4.1).
- We show that representative robustness semantics can be expressed as localized changes to the evaluator layer, supporting reusable experimentation with STL semantics rather than reimplementation (§4.2).

2 Requirements for an STL Evaluator

We begin from the common ground shared by STL tools: the standard syntax and robust semantics of STL [16,10], recalled here in the form used throughout the paper.

Definition 1 (Signal). *A* signal *over a set of variables* $\mathbf{Var}$ *is a function* $\sigma\colon \mathbb{R}_{\geq 0} \to \mathbb{R}^{\mathbf{Var}}$.

Definition 2 (STL syntax). *The set* $\mathbf{Fml}$ *of STL formulas is defined as follows:*

$$\mathbf{Fml} \ni \varphi ::= \top \mid f(\boldsymbol{x}) \geq 0 \mid \neg\varphi \mid \varphi_1 \vee \varphi_2 \mid \varphi_1 \wedge \varphi_2 \\ \mid \varphi_1 \, U_I \, \varphi_2 \mid \varphi_1 \, R_I \, \varphi_2 \mid \Diamond_I \varphi \mid \Box_I \varphi.$$

Here f is a function symbol applied to variables from $\mathbf{Var}$*, for example $f(x,y) = 4x - y - 3$, and I is a non-singular interval in $\mathbb{R}_{\geq 0}$, for example $[2,5]$, $(1,3]$, or $[0,\infty)$.*

[3] `https://github.com/midoriao/tidystl`

Informally, the robust semantics [10,8] $\rho(\varphi, \sigma, t)$ replaces the Boolean verdict with a signed margin (e.g., $\rho(x - 3 \geq 0, \sigma, t) = \sigma_x(t) - 3$), which the connectives and temporal operators aggregate by min, max, inf, and sup (e.g., $\rho(\square_{[0,2]}\,(x - 3 \geq 0), \sigma, t) = \inf_{t' \in [t,\, t+2]}(\sigma_x(t') - 3)$); the full inductive definition is recalled in Appendix A.

These definitions form the shared ground across the STL ecosystem: they are what papers cite and what tools implement. Tools built on the same definitions should, one would expect, be interchangeable at the evaluation interface: the same formula and the same signal should yield the same robustness, and a specification accepted by one tool should be accepted by another. In practice, these expectations may fail: Table 1 collects the decision points behind the divergence, and §4.1 isolates them on real tools and traces what the disagreement does downstream.

Table 1. Decision points behind cross-tool divergence, divided by origin into three groups. In the last column, C indicates an effect on *coverage* (whether a (φ, σ) pair is accepted), R indicates an effect on the reported *robustness value*, and CR indicates an effect on both.

Decision point	Effect
1) Fragments or corner cases of the standard def.	
Support for until and release operators	C
Open vs. closed interval endpoints	CR
Robustness of $\top$ and $\bot$ as $\pm\infty$	CR
2) Choices the definition leaves open	
Signal interpretation	CR
Range of expressible atomic predicates	C
Robustness of atomic predicates	R
3) Domain-specific extensions	
Equality predicates	CR
Past-time operators	C
Differentiable robustness [15]	C
Online evaluation [4]	C

Table 2. Choices bundled in the signal interpretation: how a finite timed state sequence is read as a signal in the sense of Definition 1. None is determined by the STL formula; see Appendix A.

Choice	What it fixes	Example variants
Signal model	values between samples	piecewise-linear / -constant / discrete
Window alignment	edges between samples	exact / snap to sample
Boundary rule	window past the trace end	clamp / pessimistic (empty)
Terminal step	the final sample	as-is / extend penultimate

The most consequential decision point in Table 1 is the *signal interpretation*. A signal in the sense of Definition 1 is a function on $\mathbb{R}_{\geq 0}$, but what an evaluator receives is a finite *timed state sequence* $(t_0, \boldsymbol{v}_0), \ldots, (t_n, \boldsymbol{v}_n)$ of timestamped samples, and bridging the finite representation to the idealized domain is left entirely to the tool. The bridge is not one decision but several intertwined ones, summarized in Table 2.

The tool landscape. Representative tools differ along exactly these axes: signal model, online mode, primary target application, differentiability. The pattern mirrors Table 1; a detailed tool-by-tool comparison of Breach, RTAMT, MoonLight, and STLCG++ is given in Table 5 in the appendix. In particular, the *domain-specific extensions*—the third group of decision points— represent divergence to be respected rather than erased: applications genuinely demand more than the definition provides.

Requirements. The aim of this paper is accordingly *not* to standardize the semantics but to keep robustness evaluation *interoperable*: the variation remains, but becomes explicit, comparable, and switchable. This leads to the three requirements for an evaluator layer: it should 1) *cover the standard definitions*: common specifications in accordance with Definition 2 should be expressible; 2) *admit variation as configuration*: the open choices and the domain-driven extensions of Table 1 should be backend choices or extensions below a stable evaluator interface; and 3) *make every such choice explicit* and inspectable at the interface, so that when two evaluations disagree, the responsible choice can be named.

3 Design and Implementation

This section turns the requirements of §2 into implementation structure. tidySTL exposes robustness evaluation through a small interface, while placing the choices that vary across use-cases behind a backend boundary. The key implementation question is therefore where the boundary is drawn: which parts are shared by all evaluators, and which parts are allowed to vary.

3.1 Evaluation Interface

tidySTL has two explicit inputs: formula and signal. A *formula* is a typed syntax tree, parsed from the textual STL front end. A *signal* contains one or more named, batched value sequences over a shared time grid.

An optional third input, the *backend*, encapsulates the semantic choices and the execution strategy. Evaluation takes the form `robustness(formula, signal, backend)` and returns an array of robustness values over time:

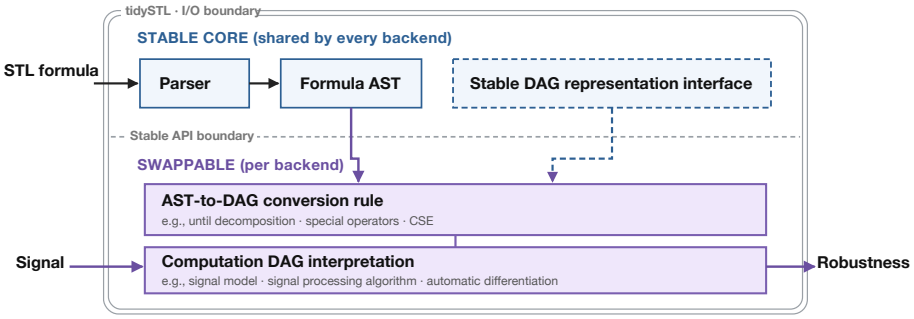

Fig. 1. Architecture of tidySTL. The stable core (parser, formula AST, DAG representation interface, public API) is shared by every backend; the AST-to-DAG conversion rule and the DAG interpretation are swappable per backend.

```python
from tidystl import Signal, parse, robustness
import numpy as np

phi = parse("G[0,5](F[0,2](velocity >= 10))")

times = np.linspace(0, 10, 200)
velocity_batch = np.array([np.sin(times), np.cos(times)])  # two traces
sig = Signal.from_dict(
    times=times,
    values={"velocity": velocity_batch},
)
rho = robustness(phi, sig)                    # default backend
rho = robustness(phi, sig, backend="breach")  # Breach-compatible
```

Keeping this interface small serves two purposes. For users, the same formula and signal can be evaluated under different documented backends without changing the surrounding application. For backend authors, new evaluation behavior attaches below a stable interface rather than requiring a new parser, signal representation, or calling convention.

3.2 Architecture

Internally, tidySTL separates a stable core from backend-specific evaluation logic, as shown in Figure 1. The stable core is shared by every backend, and it does not decide how (φ, σ) maps to a robustness value. Those decisions are delegated to the backend layer.

In the backend layer, the *semantic choices* fix what is computed: each backend resolves the decision points of Table 1 in a documented way, from the signal-interpretation choices of Table 2 to extensions such as smooth aggregation [19]. The *execution strategy* fixes how it is computed: eager interpretation, batched vectorized evaluation, caching of shared nodes, or a future incremental schedule.

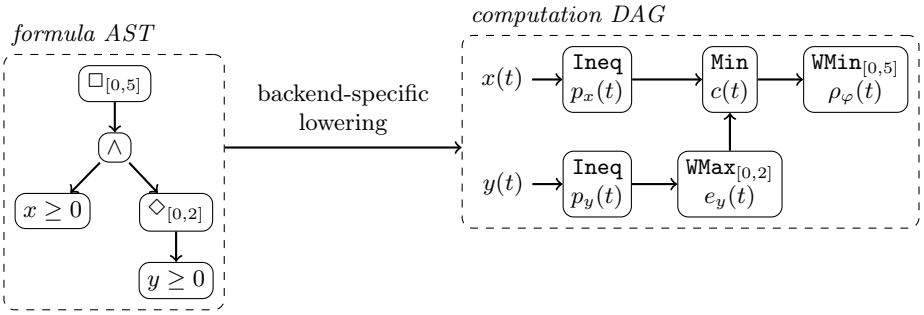

Fig. 2. Lowering of $\varphi :\equiv \square_{[0,5]}\big((x \geq 0) \wedge \diamond_{[0,2]}(y \geq 0)\big)$ from the formula AST (left) to a computation DAG (right). Nodes are typed primitive operations (`Min` is pointwise; `WMax`/`WMin` aggregate over the annotated window); edges carry the intermediate robustness signals, which remain inspectable in the evaluation result.

The separation is in service of *observability*. When the formula representation and the evaluation procedure are tightly coupled, as in most existing tools, a suspicious robustness value is difficult to diagnose: recovering which computation was actually performed means stepping through evaluator internals. Keeping the semantic choices and the execution strategy explicit makes the computation traceable from the public API down to individual primitive operations (the trace interface is documented in §B); the experiments in §4.1 and §4.2 revisit this point.

A backend realizes its choices in two steps: it lowers the formula AST into a computation *directed acyclic graph* (DAG) of typed primitive operations, and it interprets those operations into the chosen concrete signal-processing pipeline. The lowering fixes the semantic choices and the interpretation fixes the execution strategy, so semantics-preserving optimizations such as common subexpression elimination and caching of shared nodes stay on the executor side, never entangled with a semantic variant. Figure 2 illustrates the lowering on an example formula with nested temporal operators.

Extension points. A backend consists of primitive operations, a lowering rule, and an executor. Extensions may introduce new primitive operations during lowering or override the interpretation of existing ones. Programming details are given in the user manual and reference implementations, both bundled with the tidySTL repository. §4.2 measures the code required for representative variants.

Implementation status. The implementation is open to new backends and primitive operations below the stable core. The default backend supports a future-time STL core over non-uniform time grids. Additional compatibility backends are demonstrated in §4.1.

4 Evaluation

We evaluate tidySTL in two steps, one per claim. First, we show that evaluator-level choices—the decision points of Table 1—are not cosmetic: across tools, they change robustness values, verdicts, trace rankings, and downstream falsification and monitoring outcomes (§4.1). Second, we show that realizing such a choice in tidySTL is a localized change below the stable interface, at measured implementation cost and without undue overhead (§4.2).

4.1 Robustness Discrepancies Across Evaluator-Level Choices

Evaluator-level choices are the decision points of Tables 1 and 2: the questions an evaluator must answer but the formula does not. This subsection first establishes, using real-tool runs, that these choices produce observable robustness discrepancies. It then uses validated compatibility backends to localize the responsible choice and to illustrate downstream consequences. The experimental protocols and setup details are collected in Appendix D; the text here reports the findings.

Real-tool discrepancies. We first compare the original tools directly. Breach, RTAMT, and STLCG++ are run in their native environments on a shared case set, and their per-timestep robustness outputs are compared at tolerance 10^{-6}. Table 3 summarizes the result: the baseline block contains 16 operator-coverage cases from the regression suite, while the divergence block isolates one evaluator-level choice in each of five further cases.

The tools disagree even on small formulas and traces. In the baseline block, one tool deviates from the other two on 9 of the 16 cases; most disagreements come from Breach's terminal-step convention or RTAMT's boundary rule for windows extending past the trace end. The divergence block isolates further choices such as window alignment, signal model, and equality predicates. The point is not that one tool is correct and another is wrong, but that the same nominal input can induce different robustness behavior under different evaluator-level choices.

Diagnosis by validated compatibility backends. The discrepancies themselves are established by running the original tools. To inspect per-node traces and to name the responsible decision point mechanically, we built compatibility backends for each tool; each reproduces its tool's column of Table 3 per timestep (62 of 63 case–tool cells agree; the one mismatch is deliberate and documented). On a diverging case, a localizer reports the lowest syntax-tree node whose per-node traces disagree: on the boundary-rule headline row it returns `WindowMax[2,4]` vs. `DiscreteWindowMax[2,4]`, naming the boundary rule—requirement R3 of §2 executed by the tool. The validation protocol and the localization procedure are given in Appendix D.

Table 3. Cross-tool divergence matrix: real Breach 1.11.4 (B), RTAMT 0.3.5 (R), and STLCG++ 0.0.2 with exact min/max aggregation (S), compared on per-timestep robustness at absolute tolerance 10^{-6} over a trace of length $|\sigma|$. Per tool, =: matches the common behavior on the case; $*$: differs from the other two tools; –: cannot express the case (a coverage finding); †: documented special handling (RTAMT reads a non-uniform time grid as uniformly indexed samples). Each diverging row names the responsible decision point of Tables 1 and 2.

Case	$\lvert\sigma\rvert$	B	R	S	Decision point
Baseline block (operator coverage)					
$x \geq 3$	5	=	=	=	
$\neg(x \geq 0)$	5	=	=	=	
$(x \geq 0) \wedge (y \geq 0)$	5	$*$	=	=	terminal-step rule
$(x \geq 0) \vee (y \geq 0)$	5	$*$	=	=	terminal-step rule
$((x \geq 0) \wedge (y \geq 0)) \vee (z \geq 0)$	5	$*$	=	=	terminal-step rule
$\Box_{[0,2]}(x \geq 0)$	5	=	=	=	
$\Box_{[1,2]}(x \geq 0)$	3	=	$*$	=	boundary rule
$\Diamond_{[0,2]}(x \geq 0)$	5	=	=	=	
$\Diamond_{[1,3]}(x \geq 0)$	7	=	$*$	=	boundary rule
$\Diamond_{[1,2]}(x \geq 0)$	3	=	$*$	=	boundary rule
$(x \geq 0)\, U_{[0,3]}\, (y \geq 0)$	5	=	$*$	=	open vs. closed endpoints
$(x \geq 0)\, U_{[1,2]}\, (y \geq 0)$	5	=	$*$	=	open vs. closed endpoints
$(x \geq 0)\, U_{[0,4]}\, (y \geq 0)$	5	=	=	=	
$(x \geq 0)\, U_{[1,2]}\, (y \geq 0)$	2	=	$*$	=	boundary rule
$x + y \geq 0$	4	=	=	=	
$x - y \geq 0$	4	=	=	=	
Divergence block (one case per decision point)					
$\Diamond_{[2,4]}(x \geq 0)$	3	=	$*$	=	boundary rule
$(x \geq 0) \wedge (y \geq 0)$	3	$*$	=	=	terminal-step rule
$\Box_{[0.5,1.5]}(x > 0)$	3	=	–	–	window alignment (coverage)
$\Box_{[0,1]}(x > 0)$	5	=	†	–	signal model
$x = 3$	5	$*$	=	=	equality predicates

Consequences beyond robustness values. The discrepancies are not knife-edge coincidences of single constructed traces. Controlled trace sweeps show that the headline cases flip the verdicts a monitor reports and the trace rankings a search acts on over intervals of input values, not only at isolated points (sweep construction in Appendix D).

The same effect can compound downstream. We isolate this in a standard falsification loop where the system, specification, search algorithm, seeds, and budget are fixed, and only the objective semantics varies across three boundary/terminal-step rule combinations: clamp with the terminal step as-is, clamp with the extend-penultimate step, and pessimistic with as-is (setup in Appendix D). The specification's monitor horizon reaches past the end of the trace, so the boundary convention directly affects the objective. Figure 3 shows the result: the clamping variants falsify in 50/50 and 49/50 runs, while the pessimistic

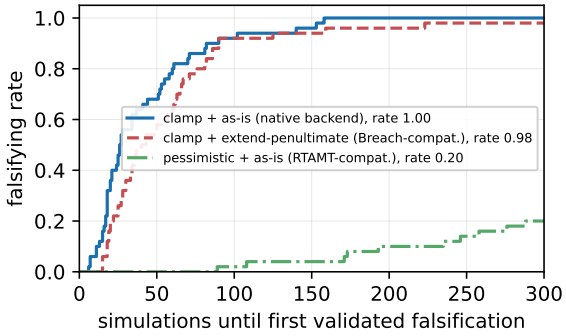

Fig. 3. Falsification performance under three objective semantics (named boundary/terminal-step rule combinations, realized as tidySTL backends), with the system, specification, search algorithm, seeds, and budget held fixed: ECDF, over 50 seeded runs, of the number of simulations until the first validated falsification (budget 300); the plateau height is the falsifying rate. Candidate violations are validated by an exact system-level check, so no backend acts as the verdict oracle.

boundary rule collapses the objective to the same out-of-range value for every input and falsifies in only 10/50 runs.

This is not evidence that one boundary rule is globally inferior. The collapse is a property of the pairing: a pessimistic boundary rule under a whole-trace monitor objective. A user who knows this convention is in play can restrict the monitor horizon and avoid the collapse. That is precisely the point: compensating for an evaluator-level choice requires knowing that the choice is there, and nothing in the optimizer's output reveals it.

4.2 Implementing Evaluation Variations with Localized Changes

The previous subsection showed that evaluator-level choices have observable consequences. We now measure the cost of realizing such choices in tidySTL: how much code changes below the stable interface, and what runtime overhead the explicit evaluator layer introduces.

Implementation effort. Table 4 shows that the shipped variants change only backend-side components. Semantic variants change primitive operations, lowering rules, or executors below the stable core; computational variants keep the semantics fixed and change only execution. Thus locality here means a boundary of modification: the parser, formula representation, signal layer, and user-facing interface are reused unchanged.

Runtime cost. Figure 4 compares tidySTL backends with real RTAMT and STLCG++ on the same workload. On single traces, the explicit evaluator layer remains within the same order of magnitude as the Python-stack references.

Table 4. Implemented variations. The key column is the layer changed; LOC is reported only to make the locality claim auditable, not as a quality metric (accounting rules in Appendix D). The full regression suite passes with all variants installed.

Variant	Layer changed	LOC	Tests
Semantic variants (what is computed)			
Breach-compatible	executor op overrides	81	53
RTAMT-compatible	own lowering + executor	191	20
STLCG++-compatible	own lowering + executor	192	20
Computational variants (how it is computed)			
STLCG++ Torch executor	executor only	241	3
Rust SIMD executor	executor only (native ext.)	109 + 347 Rust	26

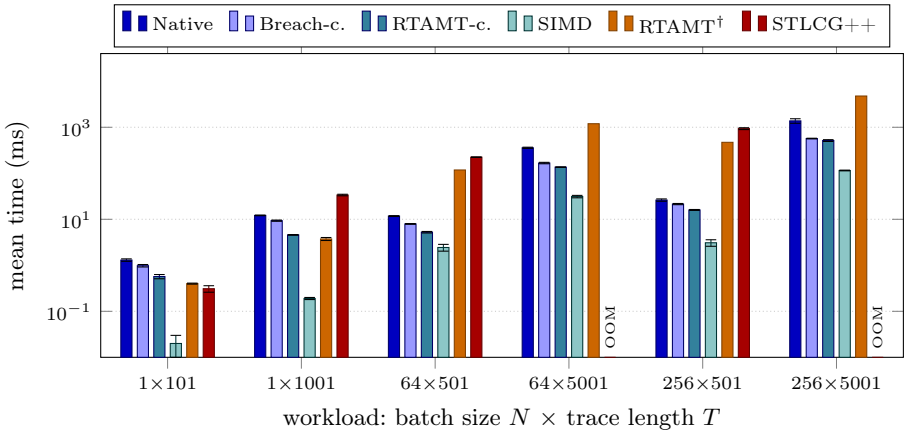

Fig. 4. Mean evaluation time on representative workloads, plotted on a log scale. All bars use the same formula $\square_{[0,5]}((x > 0) \wedge \Diamond_{[0,2]}(y > 0))$; error bars show one standard deviation. RTAMT† batch times are $N\times$ the measured per-trace time; missing STLCG++ bars exceed memory (OOM). The full timing table and benchmarking details are given in Table 6, Appendix D.

On batches, tidySTL's batch-first signal layout amortizes per-trace cost, making the native backend substantially faster than looping RTAMT and faster than the tensor-batched STLCG++ run shown here. The SIMD executor gives the expected speed ceiling once per-node observability is dropped. Thus the semantic and execution dimensions remain separable: the faster executor changes how the native semantics is computed, not what semantics is computed.

5 Related Work

§2 already positions tidySTL against the main STL tools. tidySTL differs from Breach, S-TaLiRo, RTAMT, and similar systems not by replacing their workflows but by making the evaluator layer itself the primary object of reuse [6,7,1,18,21].

Proof-oriented monitoring work [5,3] makes correctness guarantees for monitoring algorithms a central object. tidySTL does not claim comparable formal verification; instead it makes the semantic choices explicit so that divergence sources can be identified without proof.

Speaking of implicitness is not a claim that tools such as RTAMT or Breach are poorly documented. The choices in question are simply not determined by the STL formula: adopting RTAMT, for instance, commits an evaluation to its discrete-time signal and pessimistic boundary rule regardless of the formula. The ARCH-COMP 2021 report [9] documented cross-tool discrepancies arising at exactly these decision points and resolved them by post-hoc independent checking; tidySTL instead makes the responsible choices explicit and switchable in advance.

MoonLight [17] shows that pluggable monitoring domains can be a first-class contribution; tidySTL applies the same lesson to the semantic decision points of STL robustness evaluation. TeSSLa, RTLola, and HLola [13,11,12] demonstrate that front-end/back-end separation is itself a scientific contribution; their backends vary in execution platform while tidySTL's vary in the semantic choices they realize. STLCG and related differentiable tools [15,14] occupy the smooth-semantics niche that the STLCG++-compatible backend in tidySTL targets for comparison.

6 Conclusion and Future Work

tidySTL makes the semantic choices of STL robustness evaluation explicit at the evaluator boundary. By separating formula syntax, signal data, backend-specific lowering, and DAG interpretation, it exposes choices that are otherwise buried in tool internals and shows that they materially affect verdicts, trace rankings, and falsification behavior across real, widely used tools—and that the choice responsible for a disagreement can be named mechanically.

The implementation supports the view of robustness evaluation as a reusable semantic layer rather than an implementation detail of a particular monitor, falsifier, or learning pipeline. Semantic variants can be expressed by changing backend primitives, lowering rules, and DAG interpretations while preserving the public evaluation interface. Future work will extend this layer to broader semantics, including online monitoring, and expand its validation suite.

A The Decision Points in Detail

This appendix expands on the decision points of Table 1: the choices an evaluator must fix but the STL formula does not determine. We follow the three groups of the table, discussing each decision point in turn. The signal interpretation, which bundles the several choices of Table 2, is treated choice by choice, and for the two headline choices of §4.1—the boundary rule and the terminal-step rule—we show how each produces the divergence reported there. We first recall the standard robustness semantics that every evaluator approximates.

A.1 Standard Robustness

Fix a signal σ in the sense of Definition 1 and a formula $\varphi \in \mathbf{Fml}$ as in Definition 2. The standard quantitative (robustness) semantics $\rho(\varphi, \sigma, t) \in \mathbb{R} \cup \{\pm\infty\}$ is defined inductively, following [10,8]:

$$
\begin{aligned}
\rho(\top, \sigma, t) &= +\infty \\
\rho(f(\boldsymbol{x}) \geq 0, \sigma, t) &= f(\sigma_{\boldsymbol{x}}(t)) \\
\rho(\neg\varphi, \sigma, t) &= -\rho(\varphi, \sigma, t) \\
\rho(\varphi_1 \vee \varphi_2, \sigma, t) &= \max(\rho(\varphi_1, \sigma, t),\ \rho(\varphi_2, \sigma, t)) \\
\rho(\varphi_1 \wedge \varphi_2, \sigma, t) &= \min(\rho(\varphi_1, \sigma, t),\ \rho(\varphi_2, \sigma, t)) \\
\rho(\varphi_1\, U_I\, \varphi_2, \sigma, t) &= \sup_{t' \in t+I} \min\Big(\rho(\varphi_2, \sigma, t'),\ \inf_{t'' \in [t,\, t')} \rho(\varphi_1, \sigma, t'')\Big) \\
\rho(\Diamond_I \varphi, \sigma, t) &= \sup_{t' \in t+I} \rho(\varphi, \sigma, t') \\
\rho(\Box_I \varphi, \sigma, t) &= \inf_{t' \in t+I} \rho(\varphi, \sigma, t'),
\end{aligned}
$$

where $t+I = \{t+s \mid s \in I\}$ and $f(\sigma_{\boldsymbol{x}}(t))$ applies f to the values of the variables $\boldsymbol{x}$ at time t. The release operator is defined by duality, $\varphi_1\, R_I\, \varphi_2 :\equiv \neg(\neg\varphi_1\, U_I\, \neg\varphi_2)$, and the constant $\bot :\equiv \neg\top$ has $\rho(\bot, \sigma, t) = -\infty$.

Read literally, this definition assumes an idealized signal: continuous, infinitely long, and known at every instant. An evaluator never has that. It has a finite list of samples $\sigma(t_0), \ldots, \sigma(t_n)$, and to compute the sup, inf, and min above it must decide what the signal does at the times that fall between, before, and after those samples. Those decisions form the signal interpretation discussed in §A.3; the remaining decision points arise from corner cases of the definition itself (§A.2) and from extensions that applications demand beyond it (§A.4).

A.2 Fragments and Corner Cases of the Standard Definition

Support for until and release. Few tools implement the full syntax of Definition 2. The until operator, with its nested inf, is harder to implement and to vectorize than $\Box$ and $\Diamond$, and release is often dropped together with it: some tools support only the box/diamond fragment, others support until under restricted interval forms. The effect is on coverage: a specification accepted by one tool is rejected by another before any robustness is computed.

Open vs. closed interval endpoints. Definition 2 admits open and half-open intervals such as $(1, 3]$, but many front ends parse only closed ones. Where open intervals are accepted, their reading depends on the signal interpretation: under a continuous interpolant the inf over an open window coincides with that over its closure, so the difference is invisible, while under a discrete reading excluding an endpoint drops a sample and can change the aggregated value.

Robustness of $\top$ *and* $\bot$. The standard semantics assigns $\rho(\top, \sigma, t) = +\infty$ and $\rho(\bot, \sigma, t) = -\infty$. Tools differ in whether they admit infinite values at all: some propagate genuine infinities through the min/max aggregation, others substitute large finite sentinels, and a sentinel leaks into reported values once it is aggregated, scaled, or differentiated.

A.3 Choices the Definition Leaves Open

The central decision point in this group is the *signal interpretation*: the bundle of choices, summarized in Table 2, by which the finite timed state sequence an evaluator receives is read as the signal σ that the semantics quantifies over. We discuss its choices in turn, then the remaining decision points of the group.

Signal model. The signal model decides how a sampled trace is read as a function of continuous time. Under a *piecewise-linear* dense-time model (Breach, and tidySTL's native backend) the value between two samples is their linear interpolant, so a predicate such as $x \geq 0$ can change truth between sample points. A *piecewise-constant* model holds each sample until the next. A *discrete-time* model (RTAMT's default) drops the notion of between-sample time altogether: the trace is an indexed sequence and temporal operators count indices, not durations. The same formula and the same samples can therefore denote different signals before evaluation even begins.

Window alignment. Even once the signal model is fixed, a temporal window $[t + a, t + b]$ has endpoints that need not coincide with samples. The window-alignment rule decides the value taken there. A dense-time backend evaluates the interpolant at the exact endpoint; a discrete backend snaps to the nearest index or ignores the fractional part. This choice is invisible on uniformly sampled traces whose windows align with samples, which is why it is easy to overlook, and why divergence appears only on alignment-sensitive inputs.

Boundary rule. When a window $[t + a, t + b]$ reaches past the end of the trace at T, the sup or inf ranges in part over times where no sample exists. The boundary rule supplies those values. Under *clamp* (tidySTL, Breach) the last sample is treated as persisting, so an out-of-range query returns $\sigma(T)$. Under a *pessimistic* rule (RTAMT discrete) the out-of-range part contributes nothing: the window is effectively empty, so a $\Diamond$ becomes $-\infty$ (the sup of an empty set) and a $\Box$ becomes $+\infty$. This is the boundary-rule row of the divergence block in Table 3. For $\varphi = \Diamond_{[2,4]}(x \geq 0)$ evaluated at $t = 1$ on a trace that ends at $T = 2$ with $x(T) = 3$, the window $[3, 5]$ lies entirely beyond the trace. Clamp sees the persisting value 3 and reports $+3$ (satisfied); the pessimistic rule sees an empty window and reports $-\infty$ (violated). Same formula, same samples, same time point; opposite verdicts.

Terminal-step rule. At the final sample t_N a tool may report the robustness it computed there, or it may overwrite it with the value from the penultimate sample t_{N-1}. Breach does the latter by default; RTAMT does the former. The two agree whenever the final and penultimate robustness coincide, so the choice is silent on most traces. It is not silent when they differ. This is the terminal-step row of the divergence block in Table 3. For $\varphi = (x \geq 0) \wedge (y \geq 0)$ on trace A with $x = y = [5, 5, -3]$, the penultimate robustness is $+5$ and the final is -3. The standard rule reports -3; the extension rule copies the penultimate $+5$. Against a trace B that holds $+0.1$ throughout, this flips which trace looks better, reversing a ranking that a falsification search would act on.

Range of expressible atomic predicates. Definition 2 permits an arbitrary function symbol f, but front ends restrict what is expressible: to threshold comparisons on single variables, to linear arithmetic, or to a fixed expression grammar. The effect is on coverage.

Robustness of atomic predicates. Even for an accepted predicate, the evaluator must decide how it is scored: the robustness of $f(\boldsymbol{x}) \geq 0$ may be the raw signed margin $f(\boldsymbol{x})$ or a Euclidean distance to the satisfaction set [10], and the two disagree once a predicate involves several variables.

A.4 Domain-Specific Extensions

The third group extends the definition rather than reads it: applications genuinely demand more than Definition 2 provides, and each extension is a decision point because adopting a tool silently adopts its extensions along with their semantics.

Equality predicates. The equality $x = 0$ is absent from Definition 2 yet common in practical specifications. It is satisfied only at the single value $x = 0$ and admits no single standard robustness; a common choice is $-|x|$, which is never positive, so even a satisfied equality reports non-positive robustness. Breach instead scores equality by a fixed-magnitude sign convention ($\pm 10^4$), so the reported value carries no metric information at all—the equality row of Table 3 shows the resulting cross-tool divergence on inputs every tool accepts.

Past-time operators. Operators such as once and historically mirror the temporal modalities into the past. They change the direction in which an evaluator walks the trace, and tools either provide them (RTAMT) or not. The effect is on coverage.

Differentiable robustness. Gradient-based learning replaces the min/max aggregations by smooth approximations [15,19]. The smooth value deviates from the standard robustness by construction; the decision point is whether the deviation is an exposed, tunable parameter or an implicit consequence of the tool choice.

Online evaluation. Online monitoring evaluates a growing prefix, so verdicts must be issued before the trace is complete [4]. This forces a treatment of missing future information—three-valued verdicts, robustness intervals—that offline evaluation never confronts.

A.5 Why "Implicit"

None of these choices is written in the STL formula; each is fixed by the evaluator. Choosing a tool therefore silently chooses a combination of them, and two tools whose combinations differ can report different robustness, or even opposite verdicts, on identical input, as the boundary and terminal-step examples show. Nor is Table 1 a closed taxonomy: the point is not completeness but that such choices exist, are fixed by the evaluator, and should be made explicit. The contribution of tidySTL is not to declare one combination correct, but to turn the combination into an explicit, switchable argument rather than a property baked into the tool.

B The Trace Interface

This appendix records how the inspectability used in §4.1 is realized; the rationale is in §3. The facts below follow the implementation.

Every backend's `evaluate(formula, signal)` returns a result object satisfying one protocol: `robustness` (the (N, T) output array; `robustness(...)` in the public API is shorthand for it), a `has_trace` flag, and two trace methods, `trace_for(node)` and `traced_nodes()`. During lowering, the backend records for every DAG operation it emits the sub-formula node it originates from; the operation that produces a sub-formula's value is that sub-formula's *representative*. The executor evaluates the DAG in topological order and retains every operation's output, so the result carries all intermediate robustness signals, not only the root. `trace_for(sub)` returns the representative output for the sub-formula `sub`; `traced_nodes()` enumerates the sub-formulas with observable traces.

```
phi = parse("G[0,5]((x >= 0) and F[0,2](y >= 0))")
res = evaluate(phi, sig, backend="native")
sub = phi.children[0].children[1]   # the F[0,2] sub-formula
rho_sub = res.trace_for(sub)        # its (N, T) robustness trace
```

Three details matter for the experiments of §4.1. First, `trace_for` is keyed by the identity of the sub-formula node, so one parsed formula evaluated under two backends yields directly comparable per-sub-formula traces; the localizer of §4.1 is a post-order walk over `node.children` that diffs `trace_for` at each node. Second, backend post-processing participates in the trace: the Breach-compatible backend's terminal-step extension replaces the top-level output in the result, which is why the localizer reports it (the "`PointwiseMin + terminal-step extension`" line). Third, trace support is per backend: the Rust SIMD executor declares `has_trace = False` and raises on the trace methods—the per-operation retention is exactly the cost its scaling column drops.

C The Tool Landscape in Detail

Table 5 compares representative tools along the axes on which their evaluators actually differ, expanding the summary in §2.

Table 5. How representative tools differ along the axes that shape their evaluators.

Tool	Interface	Signal model	Online mode	Primary target	Auto-grad	Stack
Breach [6,7]	API + DSL	piecewise-linear	offline	falsification	no	MATLAB, mature
RTAMT [18,21]	DSL + API	discrete (default) / dense	delayed	runtime verification	no	Python, heavy
MoonLight [17]	DSL + API	spatial + temporal	partial	spatial RV	no	Java / CLI / Python
STLCG++ [14]	API	discrete (tensor)	none	differentiable learning	yes	PyTorch

D Experimental Details

This appendix records the protocols and full data behind the findings reported in §4.

Case set and comparison protocol. The baseline block comprises 16 operator-coverage cases drawn from the regression suite; the divergence block adds five cases, each targeting one decision point of Tables 1 and 2. Each tool runs in its native environment (versions as in Table 3). Outputs are normalized to per-timestep robustness vectors and compared pairwise at absolute tolerance 10^{-6} on mutually defined timesteps. STLCG++ runs with exact min/max aggregation—its default configuration; smooth aggregation is opt-in—so its column reflects its discrete-time semantics rather than smoothing, which remains a coverage-level decision point of Table 1 not exercised here.

Compatibility validation. Every tool column of Table 3 doubles as a validation check: on each case, the named compatibility backend must reproduce its tool's output per timestep at 10^{-6}—21 cases $\times$ 3 tools, hence 63 case–tool cells. The check passes on all 59 cells that carry values; three further cells match on coverage (backend and tool agree the case is inexpressible). The one mismatch is deliberate: where real RTAMT silently reinterprets a non-uniform grid as uniformly indexed samples, the RTAMT-compatible backend rejects the input instead of reproducing the reinterpretation. The backends implement each tool's *documented* rules; the reproduction check verifies the implementations rather than defining them—and it is nontrivial: on the first live Breach run it exposed two fidelity bugs in our own Breach-compatible backend (the equality convention and the window-endpoint reduction), both fixed and re-verified before the numbers reported here were produced.

Localization procedure. On the validated columns, the localizer exploits that tidySTL backends share one formula representation with per-node traces: it walks the shared syntax tree in post-order and reports the *minimal divergent*

node—the lowest node whose traces disagree while all its children agree—together with each backend's primitive operation at that node. On the two headline rows of Table 3:

```
first divergent op at ‘F[2,4]‘: WindowMax[2,4] vs
    DiscreteWindowMax[2,4] (t-index 1) -> boundary rule
first divergent op at ‘and‘: PointwiseMin vs PointwiseMin
    + terminal-step extension (t-index 2) -> terminal-step rule
```

Backend pairs whose lowerings differ structurally are compared at the shared syntax-tree level rather than between lowered representations; predicate-internal arithmetic is treated as a leaf, since no decision point lives inside it.

Trace-family construction. A single constructed trace could sit on a knife edge, so each headline case is widened to a swept family of 201 traces whose final sample varies over $[-5, 5]$, spanning regions of agreement and disagreement; no rule is taken as the reference, and the rates are symmetric between the rules by construction. For the boundary-rule family ($\Diamond_{[2,4]}(x \geq 0)$, window past the trace end), the clamping and pessimistic rules return opposite verdicts at the divergence timestep on exactly those traces whose final sample is non-negative (101 of 201). The terminal-step family is a conjunction violated only at the final sample; the as-is and extend-penultimate rules flip the final verdict on 100 of 201 traces and rank the trace oppositely against a fixed reference trace with an interior violation—the ranking signal a falsification search acts on—on 80 of 201. These rates measure the width of the disagreement region within each controlled family—the divergences are not measure-zero coincidences—not a frequency claim about any particular workload.

Falsification setup. The system is a closed-form speed model $\dot{v} = Ku(t) - c\,v$ with piecewise-constant throttle on four segments (a bounded $[0, 1]^4$ search space; traces sampled exactly, no integrator); the specification is $(v \leq 8.8) \wedge \Diamond_{[0.5,1.5]}(v \leq 25)$, whose second conjunct never fails but carries the monitor horizon past the end of the trace. A fixed (1+1)-ES with random restarts minimizes the monitor-style objective $\min_t \rho(t)$; only the objective semantics varies across arms, and every claimed falsification is validated by an exact closed-form check of the system itself, so no backend acts as the verdict oracle. The configuration, seeds, and budget (50 seeded runs, 300 simulations each) were frozen before the runs.

Per-arm results behind Figure 3: under the clamping boundary rule the search falsifies in 50/50 runs (median 26.5 simulations); adding the terminal-step extension masks violations that appear only at the final sample, costing one run and roughly 1.5× the simulations (49/50, median 40). Under the pessimistic rule the out-of-range window makes the objective $-\infty$ for every input, so the search degenerates to blind sampling: 10/50 runs falsify (median 214), and 14,027 candidate falsifications claimed across the campaign are rejected by the exact system-level check. The search runs to completion in every arm; the disagreement between the arms is silent.

Variant accounting. LOC in Table 4 counts variant-specific code only: blank, comment-only, and docstring lines are excluded, shared infrastructure is excluded, and Rust is counted separately. The native backend is the baseline and has no row; the Rust SIMD executor ships as an optional native extension alongside the five backends. The full regression suite comprises 181 tests and passes with all variants installed.

Full timing data. Table 6 tabulates the measurements plotted in Figure 4.

Table 6. Mean evaluation time (ms) $\pm$ std over a batch of N traces of length T; identical workload $\square_{[0,5]}((x > 0) \wedge \diamond_{[0,2]}(y > 0))$ for every column. This is the data plotted in Figure 4. †: RTAMT evaluates one trace per call, so batch times are $N\times$ the measured per-trace time (linearity checked at $N \in \{1, 8\}$). OOM: window materialization exceeds memory on the 32 GB host.

N	T	Native	Breach-c.	RTAMT-c.	SIMD	RTAMT	STLCG++
1	101	1.30 ± 0.08	0.98 ± 0.06	0.57 ± 0.06	0.02 ± 0.01	0.40 ± 0.01	0.31 ± 0.05
1	1001	12.15 ± 0.19	9.34 ± 0.28	4.58 ± 0.08	0.19 ± 0.01	3.74 ± 0.28	33.71 ± 1.31
64	501	11.75 ± 0.25	7.94 ± 0.15	5.24 ± 0.20	2.44 ± 0.41	$118^{\dagger}$	224 ± 3
64	5001	358 ± 11	167 ± 6	136 ± 2	31.20 ± 1.89	$1195^{\dagger}$	OOM
256	501	26.19 ± 1.54	21.42 ± 0.65	15.97 ± 0.37	3.09 ± 0.50	$471^{\dagger}$	934 ± 44
256	5001	1377 ± 155	569 ± 9	515 ± 23	115 ± 2	$4781^{\dagger}$	OOM

References

1. Annpureddy, Y., Liu, C., Fainekos, G.E., Sankaranarayanan, S.: S-TaLiRo: A tool for temporal logic falsification for hybrid systems. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS). Lecture Notes in Computer Science, vol. 6605, pp. 254–257. Springer (2011). https://doi.org/10.1007/978-3-642-19835-9_21
2. Bartocci, E., Deshmukh, J., Donzé, A., Fainekos, G., Maler, O., Ničković, D., Sankaranarayanan, S.: Specification-based monitoring of cyber-physical systems: A survey on theory, tools and applications. In: Lectures on Runtime Verification: Introductory and Advanced Topics, Lecture Notes in Computer Science, vol. 10457, pp. 135–175. Springer (2018). https://doi.org/10.1007/978-3-319-75632-5_5
3. Chattopadhyay, A., Mamouras, K.: A verified online monitor for metric temporal logic with quantitative semantics. In: Runtime Verification (RV). Lecture Notes in Computer Science, vol. 12399, pp. 383–403. Springer (2020). https://doi.org/10.1007/978-3-030-60508-7_21
4. Deshmukh, J.V., Donzé, A., Ghosh, S., Jin, X., Juniwal, G., Seshia, S.A.: Robust online monitoring of signal temporal logic. Formal Methods in System Design **51**(1), 5–30 (2017). https://doi.org/10.1007/s10703-017-0286-7
5. Dokhanchi, A., Hoxha, B., Fainekos, G.E.: On-line monitoring for temporal logic robustness. In: Runtime Verification (RV). Lecture Notes in Computer Science, vol. 8734, pp. 231–246. Springer (2014). https://doi.org/10.1007/978-3-319-11164-3_19

6. Donzé, A.: Breach, a toolbox for verification and parameter synthesis of hybrid systems. In: Computer Aided Verification (CAV). Lecture Notes in Computer Science, vol. 6174, pp. 167–170. Springer (2010). https://doi.org/10.1007/978-3-642-14295-6_17
7. Donzé, A., Ferrère, T., Maler, O.: Efficient robust monitoring for STL. In: Computer Aided Verification (CAV). Lecture Notes in Computer Science, vol. 8044, pp. 264–279. Springer (2013). https://doi.org/10.1007/978-3-642-39799-8_19
8. Donzé, A., Maler, O.: Robust satisfaction of temporal logic over real-valued signals. In: Formal Modeling and Analysis of Timed Systems (FORMATS). Lecture Notes in Computer Science, vol. 6246, pp. 92–106. Springer (2010). https://doi.org/10.1007/978-3-642-15297-9_9
9. Ernst, G., Arcaini, P., Bennani, I., Chandratre, A., Donzé, A., Fainekos, G., Frehse, G., Gaaloul, K., Inoue, J., Khandait, T., Mathesen, L., Menghi, C., Pedrielli, G., Pouzet, M., Waga, M., Yaghoubi, S., Yamagata, Y., Zhang, Z.: ARCH-COMP 2021 category report: Falsification with validation of results. In: 8th International Workshop on Applied Verification of Continuous and Hybrid Systems (ARCH21). EPiC Series in Computing, vol. 80, pp. 133–152. EasyChair (2021). https://doi.org/10.29007/xwl1
10. Fainekos, G.E., Pappas, G.J.: Robustness of temporal logic specifications for continuous-time signals. Theoretical Computer Science **410**(42), 4262–4291 (2009). https://doi.org/10.1016/j.tcs.2009.06.021
11. Faymonville, P., Finkbeiner, B., Schledjewski, M., Schwenger, M., Stenger, M., Tentrup, L., Torfah, H.: StreamLAB: Stream-based monitoring of cyber-physical systems. In: Computer Aided Verification (CAV). Lecture Notes in Computer Science, vol. 11561, pp. 421–431. Springer (2019). https://doi.org/10.1007/978-3-030-25540-4_24
12. Gorostiaga, F., Sánchez, C.: HLola: A very functional tool for extensible stream runtime verification. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS). Lecture Notes in Computer Science, vol. 12652, pp. 349–356. Springer (2021). https://doi.org/10.1007/978-3-030-72013-1_18
13. Kallwies, H., Leucker, M., Schmitz, M., Schulz, A., Thoma, D., Weiss, A.: TeSSLa – an ecosystem for runtime verification. In: Runtime Verification (RV). Lecture Notes in Computer Science, vol. 13498, pp. 314–324. Springer (2022). https://doi.org/10.1007/978-3-031-17196-3_20
14. Kapoor, P., Mizuta, K., Kang, E., Leung, K.: STLCG++: A masking approach for differentiable signal temporal logic specification. IEEE Robotics and Automation Letters **10**(9), 9240–9247 (2025). https://doi.org/10.1109/LRA.2025.3588389
15. Leung, K., Aréchiga, N., Pavone, M.: Back-propagation through signal temporal logic specifications: Infusing logical structure into gradient-based methods. In: Algorithmic Foundations of Robotics XIV (WAFR). Springer Proceedings in Advanced Robotics, vol. 17, pp. 432–449. Springer (2021). https://doi.org/10.1007/978-3-030-66723-8_26
16. Maler, O., Nickovic, D.: Monitoring temporal properties of continuous signals. In: Formal Techniques, Modelling and Analysis of Timed and Fault-Tolerant Systems (FORMATS/FTRTFT). Lecture Notes in Computer Science, vol. 3253, pp. 152–166. Springer (2004). https://doi.org/10.1007/978-3-540-30206-3_12
17. Nenzi, L., Bartocci, E., Bortolussi, L., Silvetti, S., Loreti, M.: MoonLight: A lightweight tool for monitoring spatio-temporal properties. International Journal on Software Tools for Technology Transfer **25**(4), 503–517 (2023). https://doi.org/10.1007/s10009-023-00710-5

18. Nickovic, D., Yamaguchi, T.: RTAMT: Online robustness monitors from STL. In: Automated Technology for Verification and Analysis (ATVA). Lecture Notes in Computer Science, vol. 12302, pp. 564–571. Springer (2020). https://doi.org/10.1007/978-3-030-59152-6_34
19. Pant, Y.V., Abbas, H., Mangharam, R.: Smooth operator: Control using the smooth robustness of temporal logic. In: IEEE Conference on Control Technology and Applications (CCTA). pp. 1235–1240. IEEE (2017). https://doi.org/10.1109/CCTA.2017.8062628
20. Raman, V., Donzé, A., Maasoumy, M., Murray, R.M., Sangiovanni-Vincentelli, A., Seshia, S.A.: Model predictive control with signal temporal logic specifications. In: 53rd IEEE Conference on Decision and Control (CDC). pp. 81–87. IEEE (2014). https://doi.org/10.1109/CDC.2014.7039363
21. Yamaguchi, T., Hoxha, B., Nickovic, D.: RTAMT – runtime robustness monitors with application to CPS and robotics. International Journal on Software Tools for Technology Transfer **26**(1), 79–99 (2024). https://doi.org/10.1007/s10009-023-00720-3